Module SY5 – Systèmes d'Exploitation

Dominique Poulalhon
dominique.poulalhon@irif.fr

Université Paris Cité
L3 Informatique & DL Info-Bio, Info-Jap, Math-Info
Année universitaire 2024-2025

Rappel : QCM n° 1 durant le CM de mercredi 23 octobre

Organisation du système de fichiers (fin)

Modification des répertoires

création d'une entrée de répertoire :

- avec création d'un nouvel i-nœud :

```
int open(const char *pathname, int flags, mode_t mode); /* en O_CREAT *
int creat(const char *pathname, mode_t mode);
int mkdir(const char *pathname, mode_t mode);
int mkfifo(const char *pathname, mode_t mode);
int symlink(const char *target, const char *linkpath);
```

Modification des répertoires

création d'une entrée de répertoire :

- avec création d'un nouvel i-nœud :

```
int open(const char *pathname, int flags, mode_t mode); /* en O_CREAT *
int creat(const char *pathname, mode_t mode);
int mkdir(const char *pathname, mode_t mode);
int mkfifo(const char *pathname, mode_t mode);
int symlink(const char *target, const char *linkpath);
```

- sans création d'i-nœud :

```
int link(const char *oldpath, const char *newpath);
```

Modification des répertoires

création d'une entrée de répertoire :

- avec création d'un nouvel i-nœud :

```
int open(const char *pathname, int flags, mode_t mode); /* en O_CREAT *
int creat(const char *pathname, mode_t mode);
int mkdir(const char *pathname, mode_t mode);
int mkfifo(const char *pathname, mode_t mode);
int symlink(const char *target, const char *linkpath);
```

- sans création d'i-nœud :

```
int link(const char *oldpath, const char *newpath);
```

suppression d'une entrée de répertoire

```
int unlink(const char *pathname);
int rmdir(const char *pathname);
```

modification d'une entrée de répertoire

```
int rename(const char *oldpath, const char *newpath);
```

Modification des répertoires

```
int link(const char *oldpath, const char *newpath);
```

- `oldpath` est une référence valide de fichier autre qu'un répertoire (sauf si utilisateur privilégié)
- `newpath` ne correspond à aucun lien existant,
- `dirname(newpath)` désigne un répertoire sur le même disque que `oldpath`

- crée un nouveau lien physique `basename(newpath)` dans le répertoire `dirname(newpath)` vers l'i-nœud désigné par `oldpath`,
- incrémente le compteur de liens de l'i-nœud,
- retourne 0, ou -1 en cas d'échec.

Modification des répertoires

```
int unlink(const char *pathname);
```

où `pathname` est une référence valide de fichier *autre que répertoire*,

- supprime le lien correspondant dans `dirname(pathname)`,
- décrémente le compteur de liens de l'i-nœud correspondant,
- si ce compteur est nul (et si le nombre d'ouvertures du fichier est nul), le fichier est supprimé,
- retourne 0, ou -1 en cas d'échec.

Modification des répertoires

```
int unlink(const char *pathname);
```

où `pathname` est une référence valide de fichier *autre que répertoire*,

- supprime le lien correspondant dans `dirname(pathname)`,
- décrémente le compteur de liens de l'i-nœud correspondant,
- si ce compteur est nul (et si le nombre d'ouvertures du fichier est nul), le fichier est supprimé,
- retourne 0, ou -1 en cas d'échec.

Pour la suppression des répertoires *vides* :

```
int rmdir(const char *pathname);
```

Modification des répertoires

```
int rename(const char *oldpath, const char *newpath);
```

- `oldpath` est une référence valide de fichier autre que `.` et `..`
- si `newpath` correspond à un lien existant, il doit être de même type que `oldpath`

- remplace, *de manière atomique*, le lien (déduit de) `oldpath` par le lien (déduit de) `newpath`,
- si ce lien existait déjà, il est supprimé (cf `unlink` et `rmdir`)
- retourne 0, ou -1 en cas d'échec.

Modification des répertoires

analogues avec précision d'un (descripteur de) répertoire de référence :

```
int openat(int dirfd, const char *pathname, int flags, mode_t mode);
int mkdirat(int dirfd, const char *pathname, mode_t mode);
int mkfifoat(int dirfd, const char *pathname, mode_t mode);
int symlinkat(const char *target, int newdirfd, const char *linkpath);

int linkat(int olddirfd, const char *oldpath,
                int newdirfd, const char *newpath, int flags);

int unlinkat(int dirfd, const char *pathname, int flags);

int renameat(int olddirfd, const char *oldpath,
                 int newdirfd, const char *newpath);
```

Effets des droits sur les répertoires

droit en lecture : nécessaire pour utiliser `opendir()` et `readdir()`

droit en exécution : nécessaire pour utiliser la correspondance *nom – numéro d'i-nœud*, donc utiliser le répertoire dans des références (relatives ou absolues)

droit en écriture : nécessaire pour modifier le contenu du répertoire, *i.e.* la liste de ses entrées

Apparté plus général sur la gestion des droits

chaque utilisateur dispose d'un masque *(umask)*, qui s'applique à chaque création de fichiers, ce qui lui permet de se protéger contre la création intempestive de fichiers accordant trop de droits aux autres utilisateurs

à chaque création (avec `open()`, `mkdir()`...) avec un paramètre `mode`, les droits effectivement accordés sont `mode & ~umask` pour les répertoires, et `mode & ~umask & 0666` pour les fichiers ordinaires.

```
mode_t umask(mode_t mask);
```

permet de régler l'umask, et de récupérer l'ancienne valeur

pour accorder des droits supplémentaires, il faut un appel explicite à `chmod`

Processus

Définition

processus = objet dynamique correspondant à une exécution d'un programme

Définition

processus = objet dynamique correspondant à une exécution d'un programme

au cours du temps, un processus passe en boucle par les états suivants :

- état prêt
- états actifs (actif noyau ou actif utilisateur)
- état endormi ou en attente

Définition

processus = objet dynamique correspondant à une exécution d'un programme

au cours du temps, un processus passe en boucle par les états suivants :

- état prêt
- états actifs (actif noyau ou actif utilisateur)
- état endormi ou en attente

autres états possibles :

- état transitoire initial
- état suspendu
- état zombie

un processus a deux constituants :

- son espace d'adressage : la zone mémoire où il travaille, divisée en segment de texte (le code à exécuter) et segment de données – statiques et dynamiques (pile et tas)

un processus a deux constituants :

- son espace d'adressage : la zone mémoire où il travaille, divisée en segment de texte (le code à exécuter) et segment de données – statiques et dynamiques (pile et tas)

- son bloc de contrôle : toutes les informations dont le système a besoin pour assurer la bonne gestion des processus : entre autres, identifiant, état, compteur ordinal, pointeur de pile, répertoire de travail, descripteurs...
  les informations utiles même lorsque le processus est inactif sont stockées dans la table des processus ; les autres (descripteurs par exemple) peuvent être stockées dans l'espace d'adressage.

Quelques attributs des processus

son identifiant, celui de son père

```
pid_t getpid(void);
pid_t getppid(void);
```

ses propriétaires (réel et effectif)

```
uid_t getuid(void);
uid_t geteuid(void);
int setuid(uid_t uid);
int seteuid(uid_t euid);
```

son répertoire de travail courant

```
char *getcwd(char *buf, size_t size);
int chdir(const char *path);
int fchdir(int fd);
```